

Route5 对应 Matlab 脚本的理论表述、算法流程与变量说明

目录

- Route5 对应 Matlab 脚本的理论表述、算法流程与变量说明
 - 目录
 - 摘要
 - 1. 统一记号与脚本定位
 - 1.1 本文档讨论的对象
 - 1.2 Route5 的协议层级
 - 1.3 输入、输出与目标量
 - 2. Generalized Coherent Alphabet 的理论定义
 - 2.1 相干态与复振幅
 - 2.2 字母表的生成方式
 - 2.3 为什么叫 generalized coherent alphabet
 - 3. 截断 Fock 表示与有效支持空间
 - 3.1 截断表示
 - 3.2 有效支持空间的动机
 - 3.3 投影后的本地输入态
 - 3.4 联合输入态
 - 4. IQ 测量与离散化的理论结构
 - 4.1 分束器变换
 - 4.2 双正交测量与 IQ 输出
 - 4.3 轴对齐矩形分箱
 - 4.4 Power-spaced 边界的定义
 - 5. 单模 Quadrature POVM 的数值构造
 - 5.1 连续 quadrature POVM
 - 5.2 高斯-厄米特求积
 - 5.3 节点掩码与矩阵元
 - 5.4 从 (X) 到任意相位 (θ)
 - 5.5 POVM 完备性的白化修正
 - 6. 从单模 POVM 到双模 IQ POVM
 - 6.1 张量积输出效应
 - 6.2 投影到联合支持空间
 - 7. 概率表、Raw 熵与候选输入筛选
 - 7.1 Born 概率
 - 7.2 Raw 最小熵
 - 7.3 为什么要只认证前若干个目标输入
 - 8. Single-device Guessing SDP 的理论模型
 - 8.1 SDP 的索引结构
 - 8.2 目标函数
 - 8.3 统计一致性约束
 - 8.4 完备性约束
 - 8.5 正式最小熵
 - 8.6 raw_H_min 与 formal H_min 的区别
 - 9. 程序流程与理论对象的一一对应
 - 9.1 第一步：参数解析与字母表构造
 - 9.2 第二步：构造本地截断相干态
 - 9.3 第三步：做支持压缩
 - 9.4 第四步：构造全部联合输入
 - 9.5 第五步：生成 IQ 边界与 POVM

- 9.6 第六步：生成概率表
- 9.7 第七步：raw 粗筛
- 9.8 第八步：正式 SDP 认证
- 10. 默认参数的物理意义与当前结果
 - 10.1 默认参数实例
 - 10.2 自由主线与固定光强主线
 - 10.3 为什么 Route5 能超过 2 bit
- 11. 主要变量的数学含义
 - 11.1 输入态相关
 - 11.2 联合输入相关
 - 11.3 IQ 分区相关
 - 11.4 概率与熵相关
 - 11.5 认证候选相关
- 12. 这份 Matlab 脚本与 Python 搜索器的关系
- 13. 理论边界、实验可行性与使用注意
 - 13.1 这不是实验数据版脚本
 - 13.2 实验室需要完成的工作包
 - 13.3 实验室应如何给出数据
 - 13.4 推荐的数据文件结构与必须附带的元数据
 - 13.5 所需物理设备与典型实验方法
 - 13.6 Route5 在实验上是可行的，但应视为一条新的 IQ 路线
 - 13.7 固定光强结果的真实含义与风险边界
 - 13.8 字母表规模、统计负担与认证覆盖范围
 - 13.9 主要实验风险与建议检查项
 - 13.10 截断与求积误差
 - 13.11 当前环境中的验证边界
- 14. 一句话总结

摘要

本文档面向 `guessprobprimal_route5_hybrid_iq.m` 这一份 Matlab 单文件参考脚本，采用偏理论论文的写法，对其背后的物理模型、概率生成方法、半定规划认证问题以及程序执行流程做统一说明。

这份脚本对应的是 `route5` 的“单点正式认证主流程”，而不是 Python 版搜索器的完整翻译。更准确地说，它实现的是：

1. 构造 `generalized coherent alphabet`；
2. 在截断 Fock 空间中构造相干态并压缩到有效支持空间；
3. 通过 `balanced beamsplitter + dual-homodyne / IQ coarse-graining` 生成离散输出概率；
4. 用单设备 `prepare-and-measure MDI SDP` 计算正式猜测概率；
5. 输出 `raw_H_min`、`p_guess` 与 `H_min`。

在当前脚本默认参数下，它对齐的是 Python `route5` 主线上一条已经 formal 验证过的强点：

- `cutoff = 4`
- `radii = [0.0, 0.85, 1.25]`
- `phase_values = 8` 个均匀相位
- `num_x_bins = 6`
- `num_p_bins = 2`
- `quadrature_range = 1.8`
- `boundary_gamma = 1.0`
- `max_inputs_to_certify = 3`

对应的 Python 结果文件可参考：

- `route5_local_refine_queue_mosek_v1/r0.0000_0.8500_1.2500.json`
- `route5_fixed_intensity_080160_scale120.json`

从项目结果看，前者对应的 formal 结果约为

$$H_{\min} \approx 2.11639,$$

后者对应固定光强主线时的 formal 结果约为

$$H_{\min} \approx 2.10102.$$

因此，guessprobprimal_route5_hybrid_iq.m 的理论意义，不是“随意构造一个新的 Matlab 脚本”，而是把 route5 已经在 Python 中验证过的核心协议，压缩成一份更便于导师和实验室直接检查的 Matlab 主流程版本。

同时需要明确：当前 route5 给出的

$$H_{\min} > 2$$

仍属于“理论模型 + 数值认证”结论，而不是“当前实验已经闭环完成”的结论。更准确地说，route5 更像一条独立的 IQ / dual-homodyne 新实验路线：它的认证层已经清楚，但若闭环成实验结果，还需要 route5 自己的二维 IQ / histogram / coarse-grained $P_{\mathrm{exp}}(c|x,y)$ 数据，以及把固定光强版中的半径映射关系改成实验独立标定，而不是继续作为搜索参数。

1. 统一记号与脚本定位

1.1 本文档讨论的对象

本文只讨论以下脚本：

- guessprobprimal_route5_hybrid_iq.m

不讨论 Python 搜索器如何大范围扫描 alphabet 和 IQ partition 。这些外围调度逻辑主要在：

- hybrid_iq.py
- main.py
- refine_queue.py
- intensity_menu_search.py

本文的核心目标是回答三个问题：

1. 这份 Matlab 脚本到底在解什么数学问题；
2. 它的每个主要程序块分别对应什么物理对象；
3. 脚本里的变量赋值应当如何理解。

1.2 Route5 的协议层级

route5 可以看成“连续变量前端 + 离散输出认证”的混合协议。其结构不是纯粹的离散变量方案，也不是从头到尾的纯连续变量安全证明，而是如下四层组合：

1. 可信输入层：一组受信任的相干态字母表；
2. 物理测量层：分束器后的双正交连续变量测量；
3. 数字离散化层：在 IQ 平面上做轴对齐分箱；
4. 安全认证层：对离散输出概率做单设备 MDI SDP。

因此，route5 的最终认证对象虽然是离散输出，但其概率不是“手工指定”的，而是从一个连续变量测量模型出发生成的。

通俗理解：可以把 route5 想成“先做一次高速的连续变量测量，得到很多 IQ 散点；再把这些散点按区域编号；最后不再直接研究散点本身，而是研究每个编号出现的概率，并对这些编号做随机性认证”。

1.3 输入、输出与目标量

设本地可信字母表为

$$\mathcal{A} = \{|\alpha_1\rangle, \dots, |\alpha_L\rangle\},$$

其中每个 $\alpha_\ell \in \mathbb{C}$ 是一个相干态振幅。脚本里：

- `local_alphas` 对应 $\{\alpha_\ell\}_{\ell=1}^L$ ；
- `num_local_states` = `L`。

双边输入采用笛卡尔积，因此联合输入索引记为

$$s = (x, y), \quad x, y \in \{1, \dots, L\}.$$

对应联合输入态为

$$\rho_{xy} = |\alpha_x\rangle\langle\alpha_x| \otimes |\alpha_y\rangle\langle\alpha_y|.$$

脚本里：

- `joint_states` 存放所有 ρ_{xy} ；
- `labels` 记录每个联合输入对应的零基标签 `[x-1, y-1]`；
- `num_inputs` = `L^2`。

中央测量的离散输出记为

$$c \in \mathcal{C} = \{1, \dots, C\},$$

其中

$$C = N_x N_p,$$

而脚本中：

- `num_x_bins` = `N_x`
- `num_p_bins` = `N_p`
- `num_outputs` = `C`

最终进入安全证明的观测概率表写为

$$P(c \mid x, y).$$

在脚本中，这就是 `probabilities(s,c)`。

正式认证的目标量是最大猜测概率

$$p_{\text{guess}},$$

及相应的最小熵

$$H_{\min} = -\log_2 p_{\text{guess}}.$$

通俗理解：这一节其实只是在回答三个最基础的问题：“我们往设备里送什么态”“设备会吐出什么离散结果”“最后到底想优化哪个数字”。其中 $H_{\min}$ 越大，表示攻击者越难猜中输出。

2. Generalized Coherent Alphabet 的理论定义

2.1 相干态与复振幅

在单模 Fock 空间中，相干态定义为

$$|\alpha\rangle = e^{-|\alpha|^2/2} \sum_{n=0}^{\infty} \frac{\alpha^n}{\sqrt{n!}} |n\rangle.$$

这里的 α 是复振幅，可以写成极坐标形式

$$\alpha = r e^{i\phi},$$

其中：

- $r = |\alpha|$ 是振幅模长；
- $\phi = \arg(\alpha)$ 是相位；
- 平均光子数是

$$\mu = |\alpha|^2 = r^2.$$

因此，脚本中的：

- `radii` 对应一组候选模长 r ；
- `phase_values` 对应一组候选相位 ϕ ；
- `alpha_values` 或 `local_alphas` 对应真正进入计算的 $\alpha = r e^{i\phi}$ 。

通俗理解：如果把相空间想成平面，那么 α 就是平面上的一个点；`radii` 决定这个点离原点多远，`phase_values` 决定它朝哪个方向。平均光子数 μ 本质上就是“离原点的距离平方”。

2.2 字母表的生成方式

脚本支持两种输入方式。

第一种是直接给定复振幅列表：

$$\{\alpha_1, \dots, \alpha_L\}.$$

第二种是由半径网格与相位网格生成：

$$\alpha_{m,n} = r_m e^{i\phi_n}, \quad r_m \in \mathcal{R}, \phi_n \in \Phi.$$

然后去掉重复值，得到最终字母表。

程序上对应：

- `build_alpha_values_from_grid(radii, phase_values)`
- `deduplicate_alphas(alpha_values)`

其数学作用可以概括为

$$\mathcal{A} = \text{unique}(\{r e^{i\phi} : r \in \mathcal{R}, \phi \in \Phi\}).$$

通俗理解：这一步可以理解为“先挑几圈半径，再在每一圈上挑几个角度，把这些点全列出来，重复的删掉”，最后得到真正送进协议的一组测试态。

2.3 为什么叫 generalized coherent alphabet

在很多更传统的 CV 离散化方案里，输入态往往固定在：

- 单一半径；
- 少数几个等角相位；
- 或者单一物理自由度。

`route5` 则允许：

1. 多个半径；
2. 多个相位；
3. 最终以一组离散的相关态字母表进入同一个有限维安全模型。

因此它不是“只在一个圆环上取几个相位点”，而是更一般地允许多半径、多相位的 `coherent alphabet`。这就是 `generalized coherent alphabet` 的含义。

3. 截断 Fock 表示与有效支持空间

3.1 截断表示

Matlab 脚本不在无限维 Hilbert 空间中直接计算，而是在 Fock 截断维数

$$d = \text{cutoff}$$

下工作。于是相干态被替换为截断态

$$|\alpha; d\rangle = \frac{1}{\mathcal{N}_d(\alpha)} \sum_{n=0}^{d-1} e^{-|\alpha|^2/2} \frac{\alpha^n}{\sqrt{n!}} |n\rangle,$$

其中归一化因子

$$\mathcal{N}_d(\alpha) = \left(\sum_{n=0}^{d-1} e^{-|\alpha|^2} \frac{|\alpha|^{2n}}{n!} \right)^{1/2}.$$

脚本中对应函数为：

- `build_truncated_coherent_density(alpha, M)`

它输出三样对象：

1. 截断 ket: `ket`
2. 截断密度矩阵: `rho_i = ket * ket'`
3. 其对角线: `rho_diag_i`

不过对于 `route5`，真正进入后续支持压缩和 SDP 的是完整密度矩阵，而不是仅仅对角线。

通俗理解： `cutoff` 可以粗略理解为“我们只保留 Fock 空间前 d 层来做数值计算”。这像是把无限长的展开式截成前面几项来近似，只要主要概率质量都落在前几层，近似就会比较稳。

3.2 有效支持空间的动机

虽然每个输入态先在 d 维截断空间中表示，但所有本地输入态通常只张成一个较小的子空间。若把这些截断态矢量记为

$$|\psi_1\rangle, \dots, |\psi_L\rangle,$$

则可以定义它们张成的支持子空间

$$\mathcal{H}_{\text{supp}} = \text{span}\{|\psi_1\rangle, \dots, |\psi_L\rangle\}.$$

Matlab 脚本通过 SVD 求得该子空间的正交归一基

$$B_{\text{loc}} \in \mathbb{C}^{d \times r},$$

其中 r 是局部支持秩。

程序上对应：

- `support_basis_from_vectors(local_kets)`

这样做的理论意义是：

1. 不改变这些输入态之间的 Gram 结构；
2. 把后续 SDP 维数从原始截断维数 d 压缩到真实可达的支持维数 r ；
3. 避免在“根本不会被输入态访问到”的方向上浪费变量。

通俗理解： 这里是在做一次“瘦身”。虽然我们一开始把每个态都写在 d 维空间里，但这些态真正活动的方向往往没有那么多，所以可以先找出它们真正用到的那一小块空间，再在那里面解 SDP，会省很多算力。

3.3 投影后的本地输入态

对每个输入态 ρ_x ，脚本计算其在支持基上的投影

$$\tilde{\rho}_x = B_{\text{loc}}^\dagger \rho_x B_{\text{loc}}.$$

对应程序：

- `project_density_to_basis(rho, basis)`

于是本地输入态由 ρ_x 变成 $\tilde{\rho}_x$ ，但对输入态之间的相对几何关系而言，这是精确保留的。

3.4 联合输入态

双边联合输入采用张量积：

$$\tilde{\rho}_{xy} = \tilde{\rho}_x \otimes \tilde{\rho}_y.$$

脚本中对应：

```
joint_states{row_counter} = kron(reduced_local_states{x_idx}, reduced_local_states{y_idx});
```

若局部支持维数为 r ，则联合空间维数变为

$$r_{\text{joint}} = r^2.$$

联合支持基是

$$B_{\text{joint}} = B_{\text{loc}} \otimes B_{\text{loc}}.$$

其后，所有中央测量 POVM 也会投影到这一联合支持上。

4. IQ 测量与离散化的理论结构

4.1 分束器变换

设两路输入模式的湮灭算符分别为 a 和 b 。50:50 平衡分束器由么正算符

$$U_{\text{BS}} = \exp \left[\frac{\pi}{4} (a^\dagger b - ab^\dagger) \right]$$

实现。

脚本里对应函数：

- `balanced_beamsplitter_unitary_route5(dimension)`

其内部通过产生算符和湮灭算符矩阵构造生成元，再调用矩阵指数 `expm` 得到 U_{BS} 。

4.2 双正交测量与 IQ 输出

`route5` 的中央测量不是一个简单的单模离散点击器，而是：

1. 输入的双模联合态先经过分束器；
2. 一路测 X ；
3. 另一路测 P ；

4. 得到连续输出 (x, p) 。

因此，理想连续 POVM 可以形式化写成

$$M(x, p) = U_{\text{BS}}^\dagger (|x\rangle\langle x| \otimes |p\rangle\langle p|) U_{\text{BS}}.$$

脚本不直接处理连续值 (x, p) ，而是对其做离散分箱。

通俗理解： 设备原本会给出一个连续坐标点 (x, p) 。但为了后面的离散随机性认证，我们不直接记这个精确坐标，而是只记它落进了哪个小格子里。

4.3 轴对齐矩形分箱

设 X 方向边界为

$$-\infty = b_0^{(x)} < b_1^{(x)} < \dots < b_{N_x}^{(x)} = +\infty,$$

设 P 方向边界为

$$-\infty = b_0^{(p)} < b_1^{(p)} < \dots < b_{N_p}^{(p)} = +\infty.$$

则第 (i, j) 个 IQ 矩形输出定义为

$$R_{ij} = [b_{i-1}^{(x)}, b_i^{(x)}) \times [b_{j-1}^{(p)}, b_j^{(p)}).$$

总输出数为

$$C = N_x N_p.$$

脚本中：

- `x_bounds` 和 `p_bounds` 对应上述边界；
- `output_labels(row,:) = [i-1, j-1]` 记录每个输出矩形的二维标签。

通俗理解： 这一节就是在把 IQ 平面切成若干个矩形“格子”。以后每次测量不再输出一个连续小数点，而是输出“落在第几个格子里”。

4.4 Power-spaced 边界的定义

若用户不直接给边界，脚本通过 `power_spaced_bounds(num_bins, finite_range, gamma)` 自动生成对称边界。其思路是：

先在区间 $[-1, 1]$ 上等间距取点

$$t_k = -1 + \frac{2k}{N}, \quad k = 0, \dots, N,$$

然后映射为

$$b_k = \text{sign}(t_k) |t_k|^\gamma R,$$

其中：

- `R = finite_range`
- `gamma = boundary_gamma`

最后把最外层边界替换为 $\pm\infty$ 。

于是：

- 当 $\gamma = 1$ 时，是有限边界内的等间距方案；
- 当 $\gamma > 1$ 时，边界更聚集在中心；
- 当 $\gamma < 1$ 时，边界更聚集在边缘。

这正是脚本里 `quadrature_range` 与 `boundary_gamma` 的数学含义。

通俗理解： `quadrature_range` 决定我们大致看多宽的 IQ 范围，`\gamma` 决定这些格子是“中间切得更细”还是“边上切得更细”。它们共同决定了 coarse-graining 的形状。

5. 单模 Quadrature POVM 的数值构造

5.1 连续 quadrature POVM

对单模相位为 θ 的 quadrature

$$\hat{X}_\theta = \frac{1}{\sqrt{2}} (ae^{-i\theta} + a^\dagger e^{i\theta}),$$

若第 k 个 bin 对应实轴区间 I_k ，则理想 POVM 元可写为

$$F_k^{(\theta)} = \int_{x \in I_k} |x_\theta\rangle \langle x_\theta| dx.$$

这里 $|x_\theta\rangle$ 是 $\hat{X}_\theta$ 的广义本征态。

5.2 高斯-厄米特求积

脚本不直接符号积分，而是使用高斯-厄米特数值求积。设节点与权重为

$$\{x_j, w_j\}_{j=1}^K,$$

则可把积分近似写成离散和。由于 Fock 基下的 quadrature 波函数由厄米函数给出，脚本先构造归一化厄米函数值

$$\varphi_n(x_j), \quad n = 0, \dots, d-1.$$

程序上由：

- `roots_hermite_golub_welsch(num_nodes)`
- `quadrature_hermite_data_route5(dimension, num_nodes)`

完成。

在离散化之后，一个 bin 的未修正 POVM 元可以理解为

$$F_k^{(0)} \approx \sum_{j=1}^K m_{k,j} w_j |x_j\rangle \langle x_j|,$$

其中 $m_{k,j} \in \{0, 1\}$ 表示第 j 个节点是否落在第 k 个 bin 中。

通俗理解： 高斯-厄米特求积可以理解成“用一批精心挑选的采样点和权重去近似连续积分”。也就是说，我们没有真的把整条实轴无穷细地积分，而是用少量高质量采样点把它近似出来。

5.3 节点掩码与矩阵元

脚本中 `quadrature_povms_from_bounds_route5(...)` 先根据边界构造掩码矩阵

$$M_{kj} = m_{k,j},$$

然后在 `quadrature_povms_from_node_masks_route5(...)` 中生成 base elements。

若记

$$V_{nj} = \varphi_n(x_j) \sqrt{w_j},$$

则第 k 个 base element 可以理解为

$$[F_k^{(0)}]_{nm} \approx \sum_{j=1}^K V_{nj} V_{mj} m_{k,j}.$$

这正对应了脚本中

```
masked_values = weighted_values .* sqrt(masks(idx, :));
base_elements{idx} = masked_values * masked_values.');
```

所实现的外积结构。

5.4 从 X 到任意相位 θ

Fock 基下，quadrature 旋转可由数算符相位因子给出，因此有

$$[F_k^{(\theta)}]_{nm} = e^{-i\theta(n-m)} [F_k^{(0)}]_{nm}.$$

脚本中令

$$\eta_n = e^{-i\theta n},$$

则矩阵元旋转等价于

$$F_k^{(\theta)} = (\eta \eta^\dagger) \odot F_k^{(0)},$$

其中 $\odot$ 表示逐元素乘法。

程序上对应：

```
phase = exp(-1i * theta * number_indices);
rotated{idx} = (phase * phase') .* element;
```

这正好实现了

$$(\eta_n \overline{\eta_m})_{nm} = e^{-i\theta n} e^{i\theta m} = e^{-i\theta(n-m)}.$$

5.5 POVM 完备性的白化修正

由于数值求积与截断误差，简单相加通常只有

$$S = \sum_k F_k^{(\theta)} \approx I,$$

而不一定严格等于单位算符。脚本因此引入白化修正：

$$\tilde{F}_k^{(\theta)} = S^{-1/2} F_k^{(\theta)} S^{-1/2}.$$

于是自动满足

$$\sum_k \tilde{F}_k^{(\theta)} = I.$$

程序上对应：

```
• complete_povm_via_whitening_route5(povm)
```

这是一个数值修正步骤，其目的不是改变物理模型，而是把近似积分得到的一组正半定元素重新拉回严格完备的 POVM 集。

通俗理解： 白化修正有点像“数值上的最后校准”。前面因为近似积分和截断，所有 POVM 元加起来可能不是严格的单位算符；白化这一步就是把它轻微修正一下，让整套测量重新严格满足概率完备性。

6. 从单模 POVM 到双模 IQ POVM

6.1 张量积输出效应

设 X 方向 coarse POVM 为 $\{F_i^{(x)}\}_{i=1}^{N_x}$ ，设 P 方向 coarse POVM 为 $\{F_j^{(p)}\}_{j=1}^{N_p}$ ，则分束器后的双模离散输出效应为

$$E_{ij} = U_{\text{BS}}^\dagger \left(F_i^{(x)} \otimes F_j^{(p)} \right) U_{\text{BS}}.$$

脚本中：

- `x_povms = quadrature_povms_from_bounds_route5(..., 0.0, ...)`
- `p_povms = quadrature_povms_from_bounds_route5(..., pi/2.0, ...)`
- `output_effect = kron(x_povms{x_idx}, p_povms{p_idx})`
- `povm{row_counter} = beamsplitter' * output_effect * beamsplitter`

因此，`route5` 的离散输出 `c` 本质上对应一个二维 bin (i, j) 。

通俗理解：前面我们已经把 X 轴和 P 轴各自切格子了；这里就是把“ X 落在哪格”和“ P 落在哪格”合并起来，形成最终的二维输出标签。

6.2 投影到联合支持空间

由于输入态实际只活在支持子空间 $\mathcal{H}_{\text{supp}}^{\otimes 2}$ 中，因此脚本进一步把 E_{ij} 投影成

$$\tilde{E}_{ij} = B_{\text{joint}}^\dagger E_{ij} B_{\text{joint}}.$$

程序上对应：

- `project_povm_to_basis_list(povm, joint_basis)`

这样做之后，概率仍可精确写成 Born 形式：

$$P(c | x, y) = \text{Tr} \left(\tilde{E}_c \tilde{\rho}_{xy} \right).$$

通俗理解：这一步的意思是：虽然我们前面做了支持压缩，但并没有改变真正的物理预测方式。概率仍然是“态乘测量再取迹”，只是把计算搬到了更小、更高效的子空间里。

7. 概率表、Raw 熵与候选输入筛选

7.1 Born 概率

对每个联合输入态 $\tilde{\rho}_{xy}$ 和每个离散输出 c ，脚本计算

$$P(c | x, y) = \text{Tr} \left(\tilde{E}_c \tilde{\rho}_{xy} \right).$$

程序上对应：

- `measurement_probabilities_from_states(states, povm)`

这一步得到完整概率表 `probabilities`。

7.2 Raw 最小熵

在不考虑更强兼容性约束时，对每个输入 $s = (x, y)$ 可定义一个分布级别的 raw 猜测概率：

$$p_{\text{guess}}^{\text{raw}}(s) = \max_c P(c | s).$$

相应的 raw 最小熵为

$$H_{\min}^{\text{raw}}(s) = -\log_2 \max_c P(c | s).$$

脚本中：

```
raw_h = -log2(max(max(probabilities, [], 2), 1e-15));
```

然后取其中最大者，得到

$$H_{\min, \text{best}}^{\text{raw}} = \max_s H_{\min}^{\text{raw}}(s).$$

这只是一个“哪几个输入看起来最随机”的初筛指标。

通俗理解： raw_H_min 可以理解成“只看输出分布表面上均不均匀”的快速分数。它便宜、直观，但还不是正式安全结论。

7.3 为什么要只认证前若干个目标输入

由于正式 SDP 成本高，脚本不必对全部 L^2 个联合输入都做 full certification，而是先按 raw_h 从高到低排序，再只保留前

$$K = \text{max_inputs_to_certify}$$

个候选。

程序上对应：

- `sort_target_indices_desc(raw_h)`
- `candidate_order = candidate_order(1:min(...))`

这一设计对应 Python route5 的真实 workflow：先用 raw 熵做便宜的候选筛选，再把算力集中在最有希望的 target input 上。

通俗理解： 这像是先用一次很快的预赛挑出几个好苗子，再把最贵的正式认证算力留给这些候选，而不是对所有输入都一视同仁地重算一遍。

8. Single-device Guessing SDP 的理论模型

8.1 SDP 的索引结构

对某个固定的目标输入 s^* ，脚本求解的不是一个纯粹的 classical LP，而是一个单设备 prepare-and-measure MDI SDP。

定义：

- 真实输出索引 $c \in \{1, \dots, C\}$;
- Eve 的猜测索引 $e \in \{1, \dots, C\}$;
- SDP 变量是一组厄米半正定算符

$$M_{c,e} \succeq 0.$$

程序里通过三维变量

```
variable M_ops(dimension, dimension, num_outputs * num_outputs) hermitian semidefinite
```

来承载这些算符，并用

```
op_idx = operator_index_route5(c_idx, e_idx, num_outputs);
```

实现 (c, e) 到一维切片索引的映射。

通俗理解： 这里的 $M_{c,e}$ 可以把它读成：“真实结果是 c ，而 Eve 猜的是 e 时，对应的那一块算符变量。”所以 SDP 并不是只在优化一个数字，而是在同时优化整张“真实结果/猜测结果”的算符表。

8.2 目标函数

若目标输入记为 ρ_{s^*} ，则最大猜测概率写成

$$p_{\text{guess}}(s^*) = \max \sum_{c=1}^C \text{Tr}(M_{c,c} \rho_{s^*}).$$

直观解释是：当真实输出也是 c ，而 Eve 也猜 c 时，才算猜对，因此只累加对角块 $M_{c,c}$ 。

程序中对应：

```
for c_idx = 1:num_outputs
    op_idx = operator_index_route5(c_idx, c_idx, num_outputs);
    objective_value = objective_value + real(trace(M_ops(:, :, op_idx) * rho_star));
end
maximize(objective_value)
```

通俗理解： 目标函数只把 $M_{c,c}$ 加起来，是因为只有“真实结果和 Eve 猜的一样”才算猜中。这个目标就是在问：在所有满足实验统计的量子策略里，Eve 最多能猜对多少次？

8.3 统计一致性约束

对每个输入态 ρ_s 和每个真实输出 c ，必须满足

$$\sum_{e=1}^C \text{Tr}(M_{c,e} \rho_s) = P(c | s).$$

这表示：不管 Eve 最后猜什么，真实测量落在 c 的总概率必须和观测统计一致。

程序上对应：

```
for s_idx = 1:num_inputs
    for c_idx = 1:num_outputs
        stats_sum == probabilities(s_idx, c_idx);
    end
end
```

通俗理解： 这条约束是在说：“不管 Eve 私下怎么分解和猜测，最终对外表现出来的真实统计，必须和我们观测到的概率表完全一致。” 它把 SDP 紧紧绑在实验/理论概率表上。

8.4 完备性约束

对每个猜测标签 e ，一组 $\{M_{c,e}\}_c$ 的和必须正比于单位算符：

$$\sum_{c=1}^C M_{c,e} = p_e I,$$

其中 $p_e \geq 0$ ，且

$$\sum_{e=1}^C p_e = 1.$$

这组约束对应单设备 guessing SDP 的标准结构，保证每个猜测分支本身构成一个合法的测量分解。

程序里对应：

```

variable p_e(num_outputs) nonnegative
...
complete_sum == p_e(e_idx) * identity_matrix;
sum(p_e) == 1;

```

通俗理解：可以把每个 e 看成 Eve 预先选中的一种“猜测分支”。这些分支各自对应一套合法测量，而且所有分支的权重加起来必须是 1，才能代表一个完整的物理策略。

8.5 正式最小熵

若 SDP 最优值为

$$\hat{p}_{\text{guess}},$$

则正式认证的最小熵为

$$\hat{H}_{\min} = -\log_2 \hat{p}_{\text{guess}}.$$

脚本中：

```

result_struct.p_guess = cvx_optval;
result_struct.H_min = -log2(cvx_optval);

```

这就是 route5 真正关心的最终量。

通俗理解：一旦算出 p_{guess} ，最小熵只是再做一次 $-\log_2$ 。所以本质上，整个正式认证都在围绕“把 Eve 的最优猜中率上界卡到多低”这件事展开。

8.6 raw_H_min 与 formal H_min 的区别

需要特别强调：

$$H_{\min}^{\text{raw}} \neq H_{\min}^{\text{formal}}.$$

前者只看单个输入对应的输出分布是否平坦；后者是在所有与观测统计兼容的量子攻击策略中，求一个最坏情况上仍能保证的随机性下界。

因此，脚本的工作流是：

1. 用 raw 熵找候选 target；
2. 用正式 SDP 给出真正可报告的 $H_{\min}$ 。

通俗理解：一个输入“看起来随机”，不等于它“真的已经被安全证明随机”。raw 更像肉眼观察分布图后的印象分，formal 才是把所有量子兼容性和最坏情况都算进去后的正式分数。

9. 程序流程与理论对象的一一对应

9.1 第一步：参数解析与字母表构造

脚本开始时定义：

- cutoff
- alpha_values
- radii
- phase_values
- num_x_bins
- num_p_bins
- quadrature_range
- boundary_gamma

- num_quadrature_nodes
- max_inputs_to_certify

理论上，这一步是在选定协议实例

$$\Pi = (\mathcal{A}, d, N_x, N_p, R, \gamma, K).$$

9.2 第二步：构造本地截断相干态

对每个 α_ℓ 计算：

$$\rho_\ell = |\alpha_\ell; d\rangle\langle\alpha_\ell; d|.$$

对应函数：

- build_truncated_coherent_density

9.3 第三步：做支持压缩

从 $\{\rho_\ell\}$ 对应的 ket 向量构造支持基 B_{loc} ，并投影得到 $\tilde{\rho}_\ell$ 。

对应函数：

- support_basis_from_vectors
- project_density_to_basis

9.4 第四步：构造全部联合输入

对所有 (x, y) 构造

$$\tilde{\rho}_{xy} = \tilde{\rho}_x \otimes \tilde{\rho}_y.$$

对应脚本的双重循环与 `kron(...)`。

9.5 第五步：生成 IQ 边界与 POVM

若 `x_bounds` / `p_bounds` 为空，则调用 `power_spaced_bounds` 自动生成；随后构造：

1. X 方向粗粒化 POVM；
2. P 方向粗粒化 POVM；
3. 分束器后的双模离散 POVM；
4. 投影到联合支持上的 POVM。

对应函数链：

- power_spaced_bounds
- quadrature_povms_from_bounds_route5
- quadrature_povms_from_node_masks_route5
- complete_povm_via_whitening_route5
- dual_homodyne_povm_route5
- project_povm_to_basis_list

通俗理解： 这一步是整条路线里最“物理前端”的部分。它把“怎么切 IQ 平面”真正翻译成一组可以和量子态做 Born 迹运算的 POVM 元。

9.6 第六步：生成概率表

对每个输入态、每个输出效应计算

$$P(c \mid x, y) = \text{Tr}(\tilde{E}_c \tilde{\rho}_{xy}).$$

对应函数：

- `measurement_probabilities_from_states`

9.7 第七步：raw 粗筛

计算每个联合输入对应的 raw 熵

$$H_{\min}^{\text{raw}}(x, y) = -\log_2 \max_c P(c \mid x, y),$$

然后保留前 `max_inputs_to_certify` 个候选。

9.8 第八步：正式 SDP 认证

对每个候选目标输入，调用

- `solve_single_device_guessing_route5`

求出正式 p_{guess} 与 $H_{\min}$ ，再在所有候选中选出 formal 最好的那个目标输入。

10. 默认参数的物理意义与当前结果

10.1 默认参数实例

脚本默认参数为：

$$d = 4, \quad \mathcal{R} = \{0, 0.85, 1.25\}, \quad \Phi = \left\{ \frac{2\pi k}{8} : k = 0, \dots, 7 \right\},$$

并取

$$N_x = 6, \quad N_p = 2,$$

因此总输出数为

$$C = 12.$$

边界自动生成参数为

$$R = 1.8, \quad \gamma = 1.0.$$

这是一条已经被 Python 主线 formal 验证过的 `route5` 强点配置，而不是随意给出的示例数值。

10.2 自由主线与固定光强主线

从项目现有结果看：

1. 自由搜索强点

$$\text{radii} = [0.0, 0.85, 1.25]$$

对应 formal 结果约为

$$H_{\min} \approx 2.11639.$$

2. 固定光强主线 `[0, 80, 160]`

在当前 `scale120` 映射下，对应半径近似为

$$[0.0, 0.8485, 1.2],$$

formal 结果约为

$$H_{\min} \approx 2.10102.$$

这说明：

- route5 不只是自由搜索时偶然超过 2 bit ；
- 即便把字母表收回到固定光强主线上，formal 结果仍能保持在 2 bit 以上。

但这里还要补一句关键口径：上面的“固定光强主线”并不等于“实验已经直接给出了这三个 α 值”。当前 Python 搜索器里采用的是

$$r(\mu) = r_{\max} \sqrt{\frac{\mu}{\mu_{\max}}}, \quad r_{\max} = 1.2, \quad \mu_{\max} = 160,$$

也就是把固定光强菜单 `[0,80,160]` 先映射成内部半径

$$[0, 0.8485, 1.2].$$

因此，这条结果更准确地应表述为：

- 它已经说明“在固定光强菜单约束下，route5 仍然存在 formal $H_{\min} > 2$ 的高分窗口”；
- 但若要把它升级成实验正式结果，还需要把 $\mu \mapsto \alpha$ 或 $\mu \mapsto r$ 的关系由实验独立标定固定下来，而不是继续把 `max_radius` 当成搜索参数。

通俗理解： 自由主线可以理解为“先不太管实验菜单，看看理论上最强能做到哪里”；固定光强主线则是在问“如果实验上只能用指定几个强度档位，成绩还保不保得住”。现在的答案是：还能过 2 bit ，但这一步还没有完全闭合成实验标定版。

10.3 为什么 Route5 能超过 2 bit

从模型结构上看，route5 的优势主要来自三点：

1. 输入字母表比传统单半径相位编码更丰富；
2. 中央测量不是低速单光子点击，而是高速 CV 前端；
3. 最终离散输出由二维 IQ 分区提供，因此在相同输出数下，几何结构比一维粗粒化更灵活。

当然，这并不意味着任何参数都能自动得到 $H_{\min} > 2$ 。真正起作用的是“输入几何 + IQ 分区 + SDP 约束”三者的匹配。

通俗理解： 这里没有哪一个单独参数是“万能按钮”。真正有效的是三件事一起配合得好：输入态在相空间里的摆放、IQ 平面的切格方式、以及这些统计放进 SDP 后仍然足够约束 Eve。

11. 主要变量的数学含义

下面把脚本中最关键的变量与数学对象做一次集中对应。

11.1 输入态相关

- `cutoff`
对应截断维数 d 。
- `radii`
对应相干态模长集合 $\mathcal{R}$ 。
- `phase_values`
对应相干态相位集合 Φ 。
- `alpha_values`
对应显式输入的复振幅列表 $\{\alpha_\ell\}$ 。
- `local_alphas`
对应去重后的最终本地字母表。
- `local_states`
对应每个 $\rho_\ell = |\alpha_\ell; d\rangle\langle\alpha_\ell; d|$ 。

- `local_basis`
对应本地支持子空间基 B_{loc} 。
- `reduced_local_states`
对应投影后的 $\tilde{\rho}_\ell$ 。

11.2 联合输入相关

- `joint_states`
对应全部联合输入 $\tilde{\rho}_{xy}$ 。
- `labels`
对应每个输入的 (x, y) 标签。
- `joint_basis`
对应 $B_{\text{joint}} = B_{\text{loc}} \otimes B_{\text{loc}}$ 。

11.3 IQ 分区相关

- `num_x_bins = N_x`
- `num_p_bins = N_p`
- `x_bounds`
对应 X 方向边界；
- `p_bounds`
对应 P 方向边界；
- `quadrature_range = R`
- `boundary_gamma = \gamma`

11.4 概率与熵相关

- `probabilities`
对应 $P(c \mid x, y)$ 。
- `raw_h`
对应每个输入的 raw 熵

$$-\log_2 \max_c P(c \mid x, y).$$

- `raw_best_H_min`
对应 raw 最优输入的分布级熵。
- `p_guess`
对应正式 SDP 最优值。
- `H_min`
对应正式认证熵

$$-\log_2 p_{\text{guess}}.$$

11.5 认证候选相关

- `max_inputs_to_certify`
对应正式进入 SDP 的候选输入数 K 。
- `raw_best_target`
raw 熵最高的联合输入标签。
- `certified_best_target`
formal 认证后真正最优的联合输入标签。

12. 这份 Matlab 脚本与 Python 搜索器的关系

需要明确写清：

1. 这份 Matlab 脚本实现的是 route5 的单点主流程；
2. 它并不包含 Python 中的大规模 alphabet-search 和 partition-search；
3. Python 搜索器负责“找参数”；
4. Matlab 脚本负责“给定参数后，把物理模型和正式 SDP 主流程说清楚并跑出来”。

因此，不能把“这份脚本没有整个搜索器”理解成它不完整。恰恰相反，它更适合作为导师审阅的协议核心说明文件。

通俗理解： Matlab 单文件更像“协议说明书 + 单点复现器”，而 Python 搜索器更像“大规模找好参数的工程工具”。两者分工不同，不是谁替代谁。

13. 理论边界、实验可行性与使用注意

13.1 这不是实验数据版脚本

当前脚本内部直接按物理模型生成

$$P(c \mid x, y) = \text{Tr}(E_c \rho_{xy}),$$

并不从实验文件中读取 IQ 原始数据。因此它目前对应的是：

- 理论原型；
- 数值验证；
- 与 Python 主线一致的 formal 主流程。

如果后续实验室提供真实 route5 概率数据，这份脚本的理论框架仍然成立，但“概率生成”那一段应被实验概率表替换。

这里还需要说得更具体一些。route5 需要的不是当前 route4 那种一维 Probability.mat 点击概率表，而是与 IQ / dual-homodyne 结构相匹配的数据，例如：

1. 原始 IQ 样本；
2. 二维直方图；
3. 或者至少是已经按固定分箱整理好的

$$P_{\text{exp}}(c \mid x, y).$$

因此，route5 在实验上并不是“直接复用现有 APD 数据就能完成”的小修版，而是一条需要自己数据接口的独立实验路线。现有代码在认证层结构上已经可以复用，但还没有整理成一个专门面向实验文件的整洁接入口。

通俗理解： 如果把 route4 的 Probability.mat 直接塞给 route5，那就像把“点击计数表”硬塞给一个本来想看“二维 IQ 散点图”的流程，数据格式和物理对象都对不上。

13.2 实验室需要完成的工作包

如果实验室后续要把 route5 做成真正可接入 SDP 的实验流程，至少需要把工作拆成下面几个明确的工作包。

1. 先冻结一版协议参数
需要先固定：
 - 本地 trusted alphabet 的定义；
 - 选哪些半径 / 光强；
 - 每个非零半径配几个相位；
 - IQ 分箱的 x_bounds 与 p_bounds；
 - 生成轮使用哪些目标输入，测试轮覆盖哪些输入。

2. 做输入态标定

对每个本地输入态，都需要给出它在实验中的实际标签与物理参数，例如：

- 输入编号 x 或 y ；
- 对应的强度档位或平均光子数 μ ；
- 对应的目标相位 ϕ ；
- 若已做完独立标定，还应给出复振幅 α 或等效 I/Q 位移坐标；
- 对应的不确定度或漂移范围。

若当前协议采用固定相位 coherent alphabet，则实验端不应再额外做未建模的相位随机化或相位平均，否则 trusted input 的物理含义会被改掉。

3. 做中央接收机标定

需要明确：

- 50:50 分束器是否足够接近平衡；
- 两路正交测量是否真正对应 X 与 P ；
- ADC 输出如何归一化成脚本使用的无量纲 quadrature 坐标；
- 真空参考或 shot-noise 单位是如何定义的；
- 是否做去直流、滤波、重采样、积分窗口截取。

4. 采集按输入标签分组的实验数据

对每个输入对 (x, y) ，都应能回溯到：

- 该输入对采了多少轮；
- 每轮对应的 IQ 输出是什么；
- 每轮样本是否有效；
- 数据来自哪个采集批次、哪个时间段。

5. 输出可认证的数据文件

至少要输出：

- 每个输入对、每个离散输出的计数；
- 总轮数；
- 所用分箱边界；
- 输入态定义与映射规则。

6. 做独立的 formal 认证

最后才是把实验版

$$P_{\text{exp}}(c \mid x, y)$$

代入同一套 SDP，得到实验版 p_{guess} 与 H_{min} 。

通俗理解：对实验室来说，真正要做的并不是“把一堆 IQ 数据测出来”这么简单，而是要把“输入是什么、测量怎么做、数据怎么归一化、分箱怎么定、每个概率从哪来”这整条链条一起固定住，否则后面的 SDP 没法有清晰的物理含义。

13.3 实验室应如何给出数据

从可复核性和后续认证的角度，实验数据最好按三个层级准备。

第一层是最理想的数据：逐轮原始 IQ 数据。

对每个输入对 $s = (x, y)$ ，记录每一轮的原始或归一化后样本

$$(I_t, Q_t), \quad t = 1, \dots, N_s.$$

这时固定分箱 R_c 之后，实验概率可以直接写成

$$P_{\text{exp}}(c \mid s) = \frac{1}{N_s} \sum_{t=1}^{N_s} \mathbf{1}[(I_t, Q_t) \in R_c].$$

第二层是次优但仍然很好用的数据：二维直方图计数。

若实验室不方便保存所有逐轮样本，则至少对每个输入对给出二维 histogram

$$H_s(i, j),$$

其中 (i, j) 对应 IQ 平面上的一个矩形 bin。此时

$$N_{s,c} = H_s(i, j), \quad P_{\text{exp}}(c | s) = \frac{N_{s,c}}{\sum_{c'} N_{s,c'}}.$$

第三层是最小可用数据：离散概率表加计数表。

若连二维 histogram 都不能保留，则至少应给出：

1. `counts[s,c] = N_{s,c}`
2. `totals[s] = N_s`
3. `probabilities[s,c] = N_{s,c}/N_s`
4. `x_bounds`
5. `p_bounds`
6. 输入标签 `labels[s] = (x,y)`
7. 每个本地输入对应的物理参数说明。

需要特别强调：只给一张已经归一化的概率表而不给计数，通常是不够的，因为后续若要做有限尺寸分析，就必须知道真实样本数。

13.4 推荐的数据文件结构与必须附带的元数据

为了让 route5 后续可以真正自动接入，实验室最好按“主数据 + 元数据”一起打包。

从工程实现上看，HDF5、MAT、NPZ 这类既能存多维数组又能存元数据字段的格式会比较合适。

推荐的主数据内容如下。

1. 若保存逐轮数据，则主文件至少应包含字段：
 - `shot_id`
 - `batch_id`
 - `input_x`
 - `input_y`
 - `I_raw`
 - `Q_raw`
 - `I_calibrated`
 - `Q_calibrated`
 - `valid_flag`
 - `timestamp`
2. 若保存 histogram，则主文件至少应包含：
 - `counts[x,y,i,j]`
 - `x_bounds`
 - `p_bounds`
 - `total_shots[x,y]`
3. 若只保存 coarse-grained 概率，则至少应包含：
 - `counts[x,y,c]`
 - `probabilities[x,y,c]`
 - `total_shots[x,y]`
 - 输出标签 `c <-> (i,j)` 的映射。

元数据文件则建议至少包含：

1. 输入态定义
 - 每个输入编号对应的 μ 、相位、目标 α ；
 - 这些值是“理论设定值”还是“实验标定值”；
 - 若有漂移补偿，补偿方式是什么。
2. 接收机标定
 - LO 功率或等效参考条件；
 - I/Q 通道增益；
 - 直流偏置扣除方式；
 - ADC 量程；
 - 饱和样本是否丢弃。

3. 轮次定义

- 一轮样本是如何从连续波形中截取的；
- 每轮是否做积分窗口；
- 不同轮之间是否有保护间隔；
- 是否存在 post-selection。

4. 数据处理口径

- 是否做低通或带通滤波；
- 是否做重采样；
- `x_bounds` 和 `p_bounds` 是事先固定还是事后从数据决定；
- 若有真空校准，归一化规则是什么。

这里最重要的一条原则是：只要某一步处理会改变 IQ 点云的位置或形状，就应当在元数据里明确写出来。

通俗理解：对后续程序来说，最怕的不是数据少，而是“看起来有数据，但不知道这些数是怎么处理出来的”。只要不知道某一步滤波、归一化或边界设置，后面的 formal 认证就很难说清楚。

13.5 所需物理设备与典型实验方法

从物理实现上看，`route5` 至少需要以下几类核心设备。

1. 稳定的相干光源

- 用来制备 `trusted coherent states`；
- 若协议包含多个相位点，则要求相位参考稳定；
- 若协议包含多个强度点，则要求强度菜单可重复调用。

2. 输入态调制模块

- 用于控制振幅 / 强度；
- 用于控制相位；
- 在实验上可以是独立调制器，也可以是等效的复振幅调制链路。

3. 双路输入结构

- 因为 `route5` 的联合输入是

$$|\alpha_x\rangle \otimes |\alpha_y\rangle,$$

因此实验上要么有双路独立输入支路，要么有能等效产生双路输入的时分 / 空分结构。

4. 中央 50:50 分束器

- 用于实现理论模型中的 `balanced beamsplitter`。

5. Dual-homodyne / IQ 接收机

- 可以理解为：分束器后分别测量两个正交 quadrature；
- 实验实现上，可以是两路平衡 homodyne，且参考相位差约为 $\pi/2$ ；
- 也可以是等效的 IQ coherent receiver / optical hybrid 方案。
- 无论采用哪种实现，都需要给出本振参考和正交相位关系是如何建立与维持的。

6. 低噪声探测与电子学链路

- 平衡探测器；
- 放大器；
- ADC / 示波器；
- 时钟同步与触发模块。

7. 数据采集与控制软件

- 需要能把“当前输入标签 (x, y) ”和“当前测得的 IQ 样本”正确关联起来；
- 还要能导出适合后续认证的数据文件。

典型实验方法可以按下列步骤理解。

1. 预先固定一版 `trusted alphabet`

例如先固定：

- 真空点；
- 两个非零半径层；

- 每层 8 个相位。
2. 按输入标签生成双路相干输入
对每一轮，实验控制系统决定当前使用的 (x, y) ，并在两路输入端生成相应 coherent states。
 3. 在中央做分束干涉与 IQ 读出
两路输入在 50:50 分束器上干涉，随后对两个正交 quadrature 做同步测量，得到连续 (I, Q) 或 (X, P) 样本。
 4. 对连续样本做统一的数据处理
包括：
 - 去直流；
 - 增益归一化；
 - 真空参考标定；
 - 轮次切分。
 5. 用固定分箱做 coarse-graining
按预先锁定的 x_bounds 、 p_bounds 把每轮样本映射成离散输出 c 。
 6. 汇总为实验概率表
计算每个输入对 (x, y) 下各个输出 c 的频率，得到实验版

$$P_{\text{exp}}(c \mid x, y).$$

7. 送入同一套 SDP 做正式认证
这一步原则上不改模型主线，只把“理论生成概率”替换成“实验测得概率”。

通俗理解：物理上可以把 route5 看成“先把两束调好振幅和相位的光拿去干涉，再用 IQ 接收机把结果读成二维点云，最后把点云切格子并统计每个格子的概率”。

13.6 Route5 在实验上是可行的，但应视为一条新的 IQ 路线

从物理结构上看，route5 并不是黑箱式的数值构造，而是由

相干态字母表 + 分束器 + dual-homodyne / IQ + 数字 coarse-graining + 单设备 SDP

这几步串起来的。

因此，它在实验上并非“不现实”，相反，作为一条新的连续变量前端路线，它的实验含义是清楚的：

1. 中央测量对象明确，就是 IQ / dual-homodyne；
2. 安全证明对象明确，就是 coarse-grained 后的离散输出概率；
3. 高速前端与 formal 认证可以共存，不必退回到低速 APD 点击模型。

但与此同时，也不能把它表述成“当前 route4 实验的小改版”。更准确的说法应是：

- route4 适合当前 APD / Probability.mat 主线；
- route5 更适合下一阶段专门设计的 IQ 实验主线。

13.7 固定光强结果的真实含义与风险边界

当前最贴实验限制的一条结果来自固定光强菜单

$$\{0, 80, 160\},$$

其 formal 结果约为

$$H_{\min} \approx 2.10102.$$

对应的猜测概率为

$$p_{\text{guess}} = 2^{-2.101017214340893} \approx 0.23309.$$

若以 2 bit 门槛

$$H_{\min} \geq 2 \iff p_{\text{guess}} \leq \frac{1}{4}$$

为参照，则当前余量约为

$$0.25 - 0.23309 \approx 0.01691.$$

这说明两件事：

- 1. 这条固定光强主线确实已经给出正余量，不是卡在 2 bit 门槛上；
- 2. 但这条余量也不算特别宽，因此若后续实验标定、真实噪声、有限尺寸统计或模型失配带来额外偏差，formal 值是有可能被压回 2 bit 以下的。

更关键的是，当前固定光强版仍采用

$$r(\mu) = r_{\max} \sqrt{\mu/\mu_{\max}}$$

这一搜索型映射。因此它的物理解释应当是“受固定光强菜单约束的理论高分窗口”，而不是“实验已经独立标定完毕的正式输入态设置”。后续若要做正式实验闭环，需要把 $\mu \mapsto \alpha$ 的换算独立固定下来，避免把这一步继续留在事后搜索里。

通俗理解：现在这条 2.101 bit 结果已经很有价值，因为它说明“贴着实验限制时，route5 也不是一下子就掉到 2 以下”。但它离最终实验报告还差半步，那半步就是把强度到振幅的映射单独标定清楚。

13.8 字母表规模、统计负担与认证覆盖范围

对默认强点参数

$$\mathcal{R} = \{0, 0.85, 1.25\}, \quad |\Phi| = 8,$$

由于真空态只保留一次，因此本地字母表大小为

$$L = 1 + 2 \times 8 = 17,$$

联合输入总数为

$$|\mathcal{S}| = L^2 = 289.$$

这说明 route5 的优势不是“实验工作量小”，而是“用更大的输入字母表与更丰富的 IQ 几何结构，换取 formal $H_{\min} > 2$ 的可能性”。与之对应，实验上至少有三类压力：

- 1. 输入态标定压力更大；
- 2. 测试轮与统计采样量不会轻；
- 3. 有限尺寸分析会比当前理论概率版更复杂。

还必须强调一个容易被忽略的边界：当前代码并不是对全部 289 个联合输入都做 formal 认证，而是先按 raw 指标排序，再只对前

$$K = \text{max_inputs_to_certify} = 3$$

个候选输入做正式 SDP。

因此，当前结果的准确表述应当是：

- 在当前 alphabet 与 IQ 分区下，已经找到了 formal 认证超过 2 bit 的优质目标输入；
- 但这并不等价于“所有输入对都同样提供了超过 2 bit 的正式认证随机性”。

若后续要给实验室一份真正可执行的协议说明，还应把“哪些输入用于测试、哪些输入用于正式随机数生成”预先写清楚。

通俗理解：17 个本地态、289 个联合输入，意味着 route5 换来高熵的代价之一，就是实验准备和数据采集会更重。它不是最省事的路线，而是“更复杂，但更有机会把正式熵做高”的路线。

13.9 主要实验风险与建议检查项

从实验闭环角度看，route5 的主要风险至少有以下几类。

- 1. 输入态失配风险
理论上假定的是一组已知 coherent alphabet，但实验中真正生成出来的态可能在振幅或相位上有偏差。若这一偏差没有独立标定并写进元数

据，就会破坏 trusted input 的前提。

2. 未建模相位处理风险

若理论上使用的是固定相位 alphabet，但实验端实际上做了相位漂移平均、随机相位抖动或额外相位随机化，那么进入 SDP 的 trusted states 就已经不是原来那组固定相位相干态了。

3. IQ 归一化不一致风险

若不同批次数据使用了不同增益、不同去直流口径或不同真空参考，则同样的 x_bounds / p_bounds 可能不再代表同一个物理区域。

4. LO 相位与正交性漂移风险

dual-homodyne / IQ 接收要求两个读出通道足够接近正交 quadrature。若相位差漂移较大，则实验读出的 I/Q 与理论的 X/P 对应关系会偏差。

5. 探测器非理想风险

例如：

- 分束器不平衡；
- 平衡探测器失衡；
- 电子噪声过大；
- ADC 饱和或裁剪；
- 带宽不足导致波形失真。

6. 时间漂移与样本相关性风险

若长时间采集时系统状态缓慢漂移，或者相邻轮次之间存在显著相关性，则简单频率估计未必能直接代表理想的 i.i.d. 概率模型。

7. 边界后选风险

若看到数据后再频繁调整 x_bounds 、 p_bounds ，则容易把“设计阶段搜索”与“正式认证阶段”混在一起。严格做法应当是：

- 先用探索数据决定 protocol；
- 再用独立数据集做正式报告。

8. 每输入对样本数不足的风险

因为联合输入很多，若总采样预算固定，则平均到每个输入对的轮数会下降，导致统计误差增大。

建议实验室至少做以下检查。

1. 真空输入重复测量，用来检查 IQ 零点与噪声底。
2. 固定某个输入对重复测量多个批次，用来检查漂移。
3. 给出同一批数据在不同归一化口径下的对比，确认 $P_{\mathrm{exp}}(c|x,y)$ 稳定。
4. 对容易饱和的高功率点单独做线性度检查。
5. 对最终报告使用的数据，保存未分箱的原始 IQ 样本或至少保存二维 histogram。

通俗理解： route5 的风险并不主要在“SDP 会不会跑”，而在“实验端给 SDP 的那张概率表，到底是不是和理论里说的是同一个物理对象”。只要这条数据链没钉牢，后面结果再高也会被质疑。

13.10 截断与求积误差

脚本包含两类数值近似：

1. Fock 截断 $cutoff = d$ ；
2. 高斯-厄米特求积节点数 $num_quadrature_nodes = K$ 。

因此，任何数值结果都应理解为：

给定 d 与 K 时的数值认证结果。

不过脚本已经通过：

- 支持压缩；
- POVM 白化；
- 与 Python 现有主线的一致配置

来尽量减少这些近似带来的额外误差。

通俗理解： 所有数值路线都会有“用有限计算去逼近无限理论”的问题。这里做的这些技巧，本质上就是尽量让这个近似过程更稳、更可控。

13.11 当前环境中的验证边界

本仓库当前工作流里，这份 Matlab 脚本已经完成静态对齐与文档化，但在当前终端环境中没有 Matlab/Octave 运行器，因此本文档的定位仍应是：

- 理论解释正确；
- 与 Python 主线结构对齐；
- Matlab 数值运行应在导师或实验室本机环境中进一步复核。

这并不影响本文对其理论原理与程序结构的说明。

14. 一句话总结

guessprobprimal_route5_hybrid_iq.m 的理论本质可以概括为：

generalized coherent alphabet + balanced beamsplitter + dual-homodyne + IQ coarse-graining + single-device MDI SDP.

它把 route5 中最关键的物理与安全证明主线压缩进一份 Matlab 单文件里，使导师或实验室可以不依赖 Python 搜索器，也能直接检查：

1. 输入态是怎么定义的；
2. 中央测量概率是怎么来的；
3. 离散输出是怎么构造的；
4. 正式最小熵是怎么由 SDP 算出来的。

从现有项目结果看，这条主线已经在理论数值层面给出：

$$H_{\min} > 2$$

的清晰证据；而这份 Matlab 脚本，就是对这条主线核心机制的单文件化表达。若后续要把它升级成正式实验结论，还需要再补上两件事：一是 route5 专用的真实 IQ / coarse-grained 概率数据，二是把固定光强到相干振幅的映射做成实验独立标定，而不是继续作为搜索自由度。